

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 4

Виконала:
студентка групи КН-25-1
Марусіченко А. О.
Перевірив:
доцент кафедри КІЕ
Сидоренко В. М.

Кременчук 2026

Тема: Алгоритми пошуку та їх складність

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Порядок виконання роботи

Завдання 1.

Суть лінійного пошуку полягає в тому, що ми просто по черзі перевіряємо кожен елемент масиву від початку до кінця, поки не знайдемо потрібне значення.

У найкращому випадку нам щастить, і шуканий елемент лежить на найпершому місці. Програма знаходить його з першої ж спроби, тому складність становить $O(1)$.

У найгіршому випадку все навпаки: елемента взагалі немає в масиві або він стоїть у самому кінці. Через це доводиться перевіряти абсолютно всі комірки до самого кінця, і складність зростає до $O(n)$.

Пришвидшити сам лінійний пошук неможливо, оскільки послідовний перебір завжди працює за одним принципом. Для реального підвищення швидкості на великих даних масив потрібно спочатку відсортувати, а потім використати бінарний пошук.

Завдання 2.

Суть бінарного пошуку полягає в тому, що ми шукаємо елемент у вже відсортованому масиві, постійно ділячи його навпіл і викидуючи непотрібну нам половину.

У найкращому випадку нам щастить і потрібне нам значення знаходиться точно посередині, тому складність становить $O(1)$.

У найгіршому випадку елемент або взагалі відсутній, або лишається останнім після всіх ділень, складність становить $O(\log n)$.

Завдання 3.

Алгоритм тернарного пошуку:

```
def ternary_search(arr, l, r, key):  
    if r >= l:
```

```
mid1 = l + (r - l) // 3
mid2 = r - (r - l) // 3
if arr[mid1] == key: return mid1
if arr[mid2] == key: return mid2
if key < arr[mid1]: return ternary_search(arr, l, mid1 - 1, key)
if key > arr[mid2]: return ternary_search(arr, mid2 + 1, r, key)
return ternary_search(arr, mid1 + 1, mid2 - 1, key)
return -1
```

Суть тернарного пошуку в тому, що відсортований масив ділиться двома точками на три рівні частини. Програма визначає в якій третині знаходиться елемент, і відкидає інші дві.

У найкращому випадку елемент одразу на одній із двох точок поділу, $O(1)$.

У найгіршому випадку елемент відсутній або залишився останнім після всіх ділень на три, $O(\log n)$.

Бінарний пошук є більш оптимальним за тернарний, оскільки для бінарного кількість порівнянь становить $\log_2 n$, а для тернарного – $2 \log_3 n$. Тому бінарний пошук виконує менше порівнянь на кожній ітерації та потребує менше операцій під час роботи алгоритму.

Завдання 4.

Лінійний пошук із збільшенням кількості елементів працює помітно довше, тому його краще використовувати для невеликих або невідсортованих масивів.

Бінарний пошук виконується швидше та добре підходить для великих відсортованих масивів.

Тернарний пошук теж працює швидко, але через більшу кількість перевірок трохи поступається бінарному.

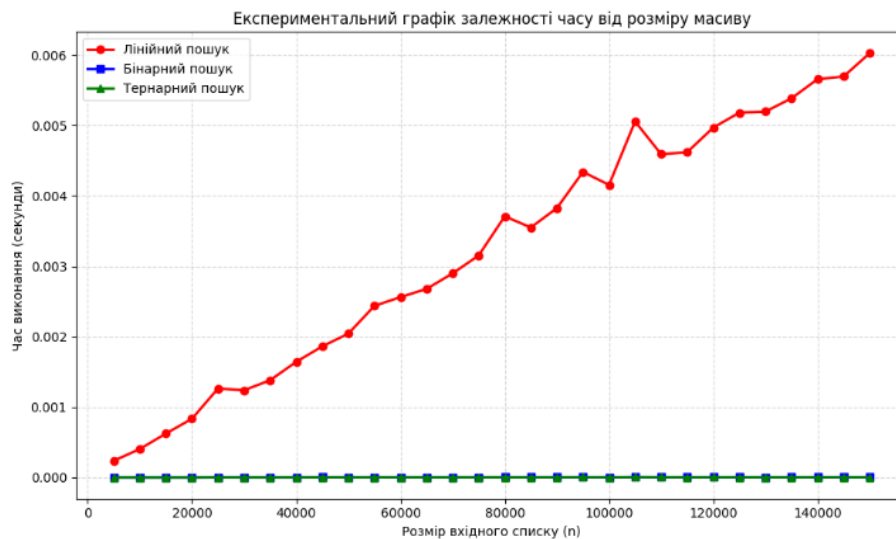


Рисунок 1 – Експериментальне дослідження

Графік показує, що лінійний пошук працює дуже повільно і його час стрімко росте з кожним новим елементом, тоді як бінарний і тернарний пошуки виконуються миттєво, тому на графіку вони просто лежать на нулі. Проте, бінарний пошук буде усе одно трохи швидший за тернарний, бо перевірок у коді він робить менше.

Завдання 5.

Лінійний пошук є універсальним, тому однаково працює як із відсортованими, так і з невідсортованими списками.

Бінарний і тернарний алгоритми ефективні лише за умови попереднього сортування масиву, без якого вони працюють некоректно й видають помилковий результат. Відповідно, для їхнього використання потрібно заздалегідь відсортувати дані, що витрачає додатковий час процесора.

Завдання 6.

Лінійний пошук підходить для маленьких або несортованих списків (наприклад, коли треба знайти товар у кошику). Сортувати невеликий масив заради одного пошуку — це просто трата часу, тому легше швидко перебрати всі елементи підряд.

Бінарний пошук — це найкращий варіант для великих баз даних, де все вже розкладено по порядку (як список студентів за ID чи слова у словнику). Оскільки масив сортується лише один раз, алгоритм потім миттєво знаходить будь-який елемент, просто ділячи дані навпіл на кожному кроці.

Тернарний пошук для звичайних масивів взагалі не вигідний, бо там забагато зайвих перевірок у коді. Його використовують у специфічних задачах чи математиці, коли треба швидко знайти мінімум або максимум на графіку функції, розбиваючи область пошуку на три частини.

Висновки: опанувала основні алгоритми сортування та навчилася методам аналізу їх асимптотичної складності.

Контрольні питання

1. Алгоритм пошуку — це набір інструкцій, який дозволяє знайти конкретне значення серед інших даних.

2. Критеріями ефективності є кількість виконаних операцій, обсяг задіяної оперативної пам'яті та складність коду.

3. Лінійний пошук здійснює простий покроковий перебір усіх елементів структури підряд.

4. Бінарний пошук щокроку зменшує область перевірки вдвічі, працюючи виключно з упорядкованими масивами.

5. Тернарний пошук розбиває простір пошуку на три однакові сегменти за допомогою двох точок поділу.

6. Головна перевага бінарного алгоритму — колосальна швидкість обробки величезних баз даних порівняно з лінійним перебором.